

Saját Vektor

A $(n \times n)$ $A \cdot v = \lambda v$
 $\uparrow$ $\uparrow$
 saját vektor saját érték

$A \cdot v = \lambda v = 0$
 egyenlet mátrix

$(A - \lambda E) \cdot v = 0 \rightarrow$ igye ha determináns $(A - \lambda E) = 0$
 karakterisztikus polinom

$A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \quad \lambda \cdot E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix}$

$A - \lambda E = \begin{pmatrix} 2-\lambda & -1 \\ -1 & 2-\lambda \end{pmatrix} \rightarrow \det(A - \lambda E) = \begin{vmatrix} 2-\lambda & -1 \\ -1 & 2-\lambda \end{vmatrix}$

Determináns számolás

$(2-\lambda)^2 - 1 = 4 - 4\lambda + \lambda^2 - 1$
 $\lambda^2 - 4\lambda + 3 = 0$
 karakter. polinom

$(\lambda - 3)(\lambda - 1) = 0$

$\lambda_1 = 3$
 $\lambda_2 = 1$ } saját érték

$\lambda_1 = 3$
 $\begin{pmatrix} 2-3 & -1 \\ -1 & 2-3 \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ -1 & -1 \end{pmatrix} \rightarrow \begin{pmatrix} -1 & -1 \\ -1 & -1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

$-v_1 - v_2 = 0$
 $-v_1 = v_2$

$v = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ normált $= \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$
 saját vektor λ_1

$\lambda_2 = 1$
 $\begin{pmatrix} 2-1 & -1 \\ -1 & 2-1 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} t_1 \\ t_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

$t_1 - t_2 = 0$
 $t_1 = t_2$

$t = 1$
 normált $= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$
 saját vektor λ_2

merőlegesek!
 szel a mátrix szimmetrikus v_1

Hatványmódszer

- legegyszerűbb eljárás, ez a legnagyobb abszolútértékű (domináns) sajátérték és a hozzá tartozó sajátvektor közelítésére szolgál
- menet

- választunk egy $x^{(0)}$ kezdővektort (pl: csupa 1-es)
- iteráció: $y^{(k)} = A x^{(k)}$
 - új irány kiszámítása (elvégezzük a mátrix szorzást)
- minden lépésben normalizáljuk a vektort, hogy ne csorduljon túl

$$\mathbf{x}^{(k+1)} = y^{(k)} / \|y^{(k)}\|$$

- euklidészi norma:

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} \\ = \sqrt{\sum x_i^2}$$

- akkor állítjuk le mikor már az előző lépés a sajátérték nem nőtt eleget, tehát pl $\lambda^{(k)} - \lambda^{(k-1)} \leq \varepsilon$, ahol pl. $\varepsilon = 0.001$ (2 tizedes pontosság kell)
- vektor irány a domináns sajátvektorhoz fog tartani
 - akkor konvergencia, ha létezik domináns sajátérték, azaz van olyan sajátértéke amire igaz, hogy $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$
 - például a $\lambda_1 = 5$ és $\lambda_2 = -5$ esetén nem konvergál
- Inverz iteráció (a mátrix inverzére alkalmazott hatványmódszer)
 - célja a legkisebb abszolútértékű sajátérték és a kapcsolódó sajátvektor közelítése

$$\lambda^{-1}x = \lambda^{-1}A\lambda^{-1}x = A^{-1}x.$$

Eszerint ha az A mátrix sajátértéke λ , és az ehhez tartozó sajátvektor x , akkor az A^{-1} mátrix egy sajátértéke λ^{-1} az x sajátvektorral. Ezen a felismerésen alapul az **inverz hatványmódszer**, vagy más néven a **Wieland-féle inverz iteráció**:

$$Ay = x^k, x^{k+1} = y / \|y\|,$$

LR és QR algoritmusok

- ezek nem csupán egy, hanem az **összes sajátérték** egyidejű meghatározására alkalmasak, **hasonlósági transzformációkra** alapul
- LR algoritmus

- a mátrixot a már említett LU felbontással felbontjuk ($A_k = L_k R_k$), majd fordított sorrendbe szorozzuk össze minden iterációban: $A_{k+1} = R_k L_k$
 - **hasonlósági transzformáció**, így nem változnak a sajátértékek
 - azért van LR mert régen R mint jobb is a felsőháromszög alakra volt értetendő, de modernebb helyzetbe inkább az U maradt meg
- minnél több iteráció megy végbe előbb utóbb egy felső háromszög alakúvá válik, ahol a főátlóban megtalálható az összes sajátérték
- (gyakorlatba nem alkalmazzuk) ha a sajátvektorokat is meg akarjuk határozni akkor iterációkba vezetni kell az A_k , mellett egy P_k -t is, a $P_0 = I$, azaz kezdetben egy egységmátrix ez, majd minden iterációban $P_{k+1} = P_k L_k$
 - végül mikor az iterációt leáll akkor a P_k oszlopai lesznek a sajátvektorok
- problémája, hogy numerikusan instabil

$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$

Új példa: Induljunk az $x_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ vektorból

1. Iteráció

- Szorzás: $y_1 = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$
- Sajátérték tipp ($\lambda^{(1)}$): A legnagyobb elem a 4.
- Normálás (q_1): $q_1 = \frac{1}{4} \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}$

2. Iteráció

- Szorzás: $y_2 = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 4.5 \\ 3.5 \end{pmatrix}$
- Sajátérték tipp ($\lambda^{(2)}$): A legnagyobb elem a 4.5.
- Normálás (q_2): $q_2 = \frac{1}{4.5} \begin{pmatrix} 4.5 \\ 3.5 \end{pmatrix} \approx \begin{pmatrix} 1 \\ 0.7778 \end{pmatrix}$

3. Iteráció

- Szorzás: $y_3 = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0.7778 \end{pmatrix} \approx \begin{pmatrix} 4.7778 \\ 4.3334 \end{pmatrix}$
- Sajátérték tipp ($\lambda^{(3)}$): A legnagyobb elem a 4.7778.
- Normálás (q_3): $q_3 = \frac{1}{4.7778} \begin{pmatrix} 4.7778 \\ 4.3334 \end{pmatrix} \approx \begin{pmatrix} 1 \\ 0.9070 \end{pmatrix}$

Eredmény

Látható, hogy a λ értékei ($4 \rightarrow 4.5 \rightarrow 4.7778 \dots$) lépésről lépésre közelednek a valódi domináns sajátértékhez, az 5-höz.

A sajátvektor tippünk (q_k) pedig közeledik a $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ vektorhoz.

• QR algoritmus

- gyakorlatban ezt használjuk (pl: matlab), mert stabil és eredménye is ortogonális
- a mátrixot QR felbontással felbontjuk ($A_k = Q_k R_k$), majd fordított sorrendbe összeszorozzuk őket: $A_{k+1} = R_k Q_k$
 - Q = ortogonális mátrix, míg R = felsőháromszög
 - és mivel Q ortogonális, azaz transzponáltja egyezik az inverzával ezért a kerekítési hibák nem erősödnek fel, a konvergencia végén a főátló tartalmazza a sajátértékeket
- minnél több iteráció megy végbe előbb utóbb egy felső háromszög alakúvá válik, ahol a főátlóban megtalálható az összes sajátérték
- (gyakorlatban nem alkalmazzuk) ha a sajátvektorokat is meg akarjuk határozni akkor iterációkba vezetni kell az A_k , mellett egy P_k -t is, a $P_0 = I$, azaz kezdetben egy egységmátrix ez, majd minden iterációban $P_{k+1} = P_k Q_k$
 - végül mikor az iterációt leáll akkor a P_k oszlopai lesznek a sajátvektorok

Gersgorin-körök

- becslési módszer, segít lokalizálni a sajátértékeket a komplex síkon a kiszámításuk nélkül
 - az összes sajátérték ezen körök egyesítésében, uniójában található
 - arra jó, hogy gyorsan megállapítható, hogy a sajátértékek mely tartományba mozognak, vagy pl igazolható, hogy egy mátrix invertálható-e (ha 0 nincs benne egyik körben se akkor invertálható)
 - illetve, ha van egy különálló kör ami másikkal nem áll metszésbe akkor annak az intervallumán belül fix van sajátérték
- minden a_{ii} főátlóbeli elem köré rajzolunk egy kört, melynek sugara a sor többi elemének abszolútértékű-összege ($R_i = \sum_{j \neq i} |a_{ij}|$)

$$A = \begin{pmatrix} 4 & 1 & 1 \\ 0.5 & 1 & 0.25 \\ 1 & -1 & -2 \end{pmatrix}$$

Határozzuk meg a három Gersgorin-kört!

1. Kör (K_1) – az első sorból:

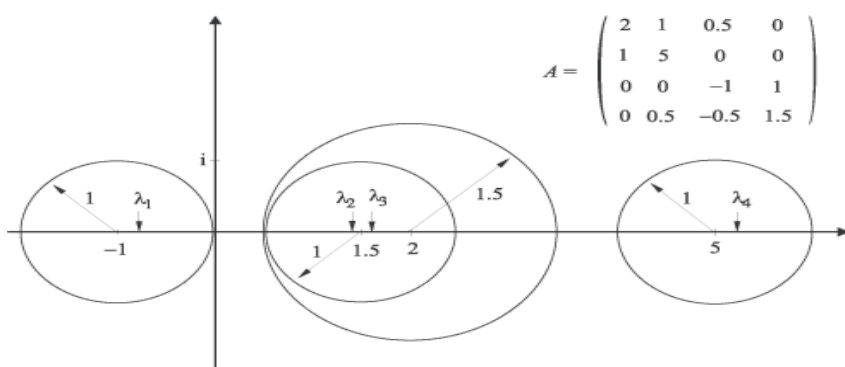
- Közepont (c_1): A főátló eleme $\rightarrow 4$
- Sugár (R_1): A többi elem abszolút értékének összege $\rightarrow |1| + |1| = 2$
- A kör: 4 középpontú, 2 sugarú kör. (A valós tengelyen a $[2, 6]$ intervallumot fedi le).

2. Kör (K_2) – a második sorból:

- Közepont (c_2): A főátló eleme $\rightarrow 1$
- Sugár (R_2): A többi elem abszolút értékének összege $\rightarrow |0.5| + |0.25| = 0.75$
- A kör: 1 középpontú, 0.75 sugarú kör. (A valós tengelyen a $[0.25, 1.75]$ intervallumot fedi le).

3. Kör (K_3) – a harmadik sorból:

- Közepont (c_3): A főátló eleme $\rightarrow -2$
- Sugár (R_3): A többi elem abszolút értékének összege $\rightarrow |1| + |-1| = 1 + 1 = 2$
- A kör: -2 középpontú, 2 sugarú kör. (A valós tengelyen a $[-4, 0]$ intervallumot fedi le).



POLINOMOK ÉS POLINOMOK ZÉRUSHELYE

- a $p(x) = a_0 x^n + a_1 x^{n-1} + \dots + a_n$ alakú polinom
 - $a_i x^{n-i}$ a tagok
 - ahol a_i az i -edik (általános esetben komplex) polinom-együttható
 - az a_0 pedig a legmagasabb fokú tag együtthatója, ez nem nulla garantáltan
 - az a_n a konstans tag
 - a polinom foka a legmagasabb hatványkitevő (itt n)
- bonyolultabb függvényeket szokás polinommal közelíteni, mivel polinomokat egyszerű deriválni és integrálni, valamint a zérushelyeiket is könnyű közelíteni
- nincs olyan megoldóképlet, mely tetszőleges polinom gyökeit meg tudná adni az együtthatókat tartalmazó, véges sok aritmetikai művelettel és gyökvonással felírt kifejezések formájában
 - numerikus nehézségek magasabb fokú polinomoknál ($n > 4$) lépnek fel
- kiértékelésére a **Horner-elrendezés** használatos:
$$p(x) = (((a_0 x + a_1)x + a_2)x + \dots + a_{n-1})x + a_n$$
 - a sémának műveletigénye n szorzás és n összeadás (alap polinomok több számítás igényel)
- programozásban a Horner elrendezéshez használható a **Ruffini sorozat**
 - $p(x)$ polinomot elosztjuk egy $(x - \hat{x})$, így $p(x) = (x - \hat{x}) * q(x) + b_n$
 - $\hat{x}$ (x kalap) egy konstans szám
 - $q(x)$ az egyel alacsonyabb foksámú polinom
 - b_n meg a maradék
 - ha a táblázat végén a maradék 0 akkor a tippelt $\hat{x}$ valósszám a gyöke, tehát $O(n)$ alatt ez tesztelhető ezzel
- **Iterált Horner-elrendezés**
 - az iterált Horner-módszer azt jelenti, hogy a kapott hányadospolinomra újra és újra alkalmazzuk a Horner-táblázatot ugyanazzal a $\hat{x}$ számmal, azaz kiszámoljuk az értékét
 - az alapja, hogy a Ruffini-sorozatból kapott $q(x)$ polinomra $p'(\hat{x}) = q(\hat{x})$
- **Zérushely (gyök)**
 - $p(x_0) = 0$, azaz a gyök, vagy zérushely geometriailag azt jelenti, hogy a polinom grafikonja x pontban metszi, vagy érinti az x tengelyt
 - x_i gyök **multiplicitása** v_i (ejtsd nú), ha $p(x)$ felírható $(x - x_i)^{v_i} q(x)$ alakba, ahol $q(x)$ olyan polinom melynek az x_i már nem gyöke
 - egy n -edfokú polinomnak a komplex számok halmazán pontosan n darab komplex gyöke van ha a multiplicitásukkal együtt számoljuk a gyököket
 - x_0 komplex szám a $p(x)$ polinom zérushelye (gyöke), akkor ha $p(x_0) = 0$, ez csak akkor igaz, mikor teljesül, hogy $p(x) = (x - x_0) q(x)$, valamely $q(x)$ polinommal (multiplicitás teljesül)
 - ha $p(x)$ valós együtthatós polinomnak van egy x_i komplex gyöke akkor annak komplex konjugáltja, $\bar{x}_i$ is gyöke (azonos multiplicitással)
 - ha ismerjük a $p(x)$ polinom gyökeit ($x_1, \dots, x_n$), akkor felírható **gyöktényező**s alakban: $p(x) = a_0 (x - x_1) (x - x_2) \dots (x - x_n)$

- Bézout tétel, ha a c szám gyöke a $p(x)$ polinomnak, akkor a polinom maradék nélkül osztható az $(x - c)$ elsőfokú polinommal (ezt nevezzük gyöktényezőnek)
- ha a $p(x)$ polinom gyökei $x_1, x_2, \dots, x_n$ akkor a $q(x) = p(-x)$ polinom gyökei a $-x_1, -x_2, \dots, -x_n$
- $r(x) = x^n p(1/x)$ polinom gyökei az $1/x_1, 1/x_2, \dots, 1/x_n$, feltéve, hogy $p(0) \neq 0$
- gyökszámítás
- az első és másodfokú polinomra még van megoldóképlet

Elsőfokú ($ax + b = 0$): $x = -\frac{b}{a}$

Másodfokú ($ax^2 + bx + c = 0$): A jól ismert megoldóképlettel:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- a harmad és negyedfokúra is (cardano- és ferrari-féle tételek), de ezek gyakorlatban bonyolultak
- ötödfokúra és magasabbra már nem is létezik általános algebrai megoldóképlet

INTERPOLÁCIÓ ÉS LAGRANGE-INTERPOLÁCIÓ

Interpolációs alapfeladat

- mivel a polinomokkal a műveletek gyorsan és hatékonyan végrehajthatók számítógépen, illetve több jó tulajdonsága is van (pl: differenciálhatóság), ezért a bonyolultabb függvények közelítésére polinomokat használunk
- **interpoláció** az a feladat, mikor adott egy (x_i, y_i) , $i = 1, 2, \dots, m$ pontsorozat és ehhez le kell előállítani azt a függvényt, mely minden ponton átmegy és legjobban közelíti azokat
 - az előállított $f(x)$ függvény a kapott argumentumokon felveszi a megfelelő kapcsolódó értéket $f(x_i) = y_i$
 - m alappont esetén ez egy legfeljebb $m - 1$ -edfokú polinom létezik, amely az adott pontokba a megadott értéket veszi fel (*tétel*)
 - figyelni kell, hogy ne legyen túlillesztés és a legtömörebbet válasszuk
- **polinom interpoláció**
 - a pontokra egy magas fokszámú polinomot illesztünk, ekkor a polinom fokszáma n , egyenlő az $m - 1$ -el, azaz eggyel kevesebb mint a pontok száma
- **racionális interpoláció**
 - az adatpontokra nem polinomot, hanem két polinom hányadosát, azaz egy racionális törtfüggvényt illesztünk
- **spline interpoláció**
 - a spline interpoláció **szakaszokra bontja a tartományt**, és minden szakaszon egy-egy alacsony fokszámú polinomot (leggyakrabban harmadfokút) használ
 - elvárt, hogy a szomszédos polinomok a csatlakozási pontokban előírt differenciálhatósági feltételeknek is eleget tegyenek
 - a fokszám lényegesen kisebb mint az alappontok száma
 - az alkalmazott polinomok fokszáma alapján van: kvadratikus és köbös spline
 - amikor olyan polinomot illesztünk melynek fokszáma kisebb, mint $m - 1$ akkor görbeillesztésről beszélünk

Lagrange-Interpoláció

- polinom interpoláció
- tehát feladata, hogy **$n + 1$ darab alappontra $(x_0, y_0), \dots, (x_n, y_n)$ illesszünk egy olyan függvényt, mely minden ponton átmegy** (polinomokkal megvalósítva), extra feltétel, hogy ezek az **alappontok páronként különbözzenek**
 - polinominterpoláció kimondja, hogy a pontok száma 1-el nagyobb, mint a polinom fokszáma (n), ezért van $m = n + 1$ alappont
 - ha az **alappontok páronként különböznek** akkor tudjuk, hogy **pontosan egy interpolációs polinom létezik**, mely az adott pontokon áthalad
- **Lagrange-alak:** a polinomokat nem közvetlen együtthatókkal hanem Lagrange-alappolinomokkal írjuk fel
 - minden alapponthoz tartozik egy saját polinom, az $L_k(x)$ alappolinomok tulajdonsága, hogy az x_k helyen 1-et az összes többi x_j alappontban 0-t vesznek fel, képletük:
$$L_k(x) = \prod_{j=0, j \neq k}^n \frac{x - x_j}{x_k - x_j}$$
 - a végeredmény (Lagrange-interpolációs polinom) az alpontokhoz tartozó saját $L_k(x)$ alappolinomok lineáris kombinációjaként jön létre:
$$L(x) = \sum_{k=0}^n y_k L_k(x)$$
 - ez egy $m - 1$ -edfokú polinom
- **előnye:** egyszerű felépítés, elméleti számításokhoz (pl: integrál közelítő szabályok levezetésére) jó
- **hátránya:** Runge-jelenség
 - bár a polinom pontosan illeszkedik a pontokra, az alappontok között nagy kilengéseket produkálhat, különösen magas fokszám (sok alappont van) esetén
 - emiatt a gyakorlatban a spline interpoláció használata gyakoribb

$$L_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j} = \frac{(x - x_1)(x - x_2) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_1)(x_i - x_2) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

A LAGRANGE INTERPOLÁCIÓ HIBÁJA

Az alappontok táblázatba rendezve:

x	-1	0	1	4
$p_3(x)$	2	0	4	0

Határozzuk meg az $L_i(x)$ polinomokat:

$$L_1(x) = \frac{(x - 0)(x - 1)(x - 4)}{(-1 - 0)(-1 - 1)(-1 - 4)}$$

$$L_2(x) = \frac{(x + 1)(x - 1)(x - 4)}{(0 + 1)(0 - 1)(0 - 4)}$$

$$L_3(x) = \frac{(x + 1)(x - 0)(x - 4)}{(1 + 1)(1 - 0)(1 - 4)}$$

$$L_4(x) = \frac{(x + 1)(x - 0)(x - 1)}{(4 + 1)(4 - 0)(4 - 1)}$$

Írjuk fel az interpoláló polinomot:

$$\begin{aligned} p_3(x) &= 2L_1(x) + 0L_2(x) + 4L_3(x) + 0L_4(x) = \\ &= 2 \frac{x^3 - 5x^2 + 4x}{-10} + 4 \frac{x^3 - 3x^2 - 4x}{-6} = \\ &= -\frac{x^3 - 5x^2 + 4x}{5} - \frac{2x^3 - 6x^2 - 8x}{3} = \\ &= \frac{-3x^3 + 15x^2 - 12x - 10x^3 + 30x^2 + 40x}{15} = \\ &= \frac{-13x^3 + 45x^2 + 28x}{15} = \\ &= -\frac{13}{15}x^3 + 3x^2 + \frac{28}{15}x \end{aligned}$$

Az interpolációs feladat megfogalmazásában eddig nem kellett semmilyen kikötést tennünk az interpolált $f(x)$ függvényre. A hibavizsgálathoz viszont feltesszük, hogy az $f(x)$ függvény $n + 1$ -szer differenciálható azon a legszűkebb $[a, b]$ intervallumon, amelyben benne van az $n + 1$ alappont, és az $\hat{x}$ interpolációs vagy extrapolációs pont is.

Vezessük be a

$$g(x) = f(x) - p_n(x) - \frac{\omega_n(x)}{\omega_n(\hat{x})}(f(\hat{x}) - p_n(\hat{x}))$$

jelölést, ahol $p_n(x)$ a Lagrange-féle interpolációs polinom, $\omega_n(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$. Ez a $g(x)$ függvény nyilván legalább $n + 1$ -szer differenciálható az $[a, b]$ intervallumon, és az $n + 1$ -edik derivált:

$$g^{(n+1)}(x) = f^{(n+1)}(x) - \frac{(n+1)!}{\omega_n(\hat{x})}(f(\hat{x}) - p_n(\hat{x})),$$

mivel a legfeljebb n -edfokú $p_n(x)$ polinom deriváltja nulla, az egy főegyütthatós $\omega_n(x)$ -é pedig $(n+1)!$.

A definíció szerint $g(x)$ minden alappontban nulla, és $g(\hat{x}) = 0$ is teljesül, tehát $g(x)$ -nek $[a, b]$ -ben legalább $n + 2$ különböző zérushelye van. A kalkulusban megismert Rolle tétel szerint tetszőleges két zérushely között van olyan pont, ahol a $g'(x)$ derivált is felveszi a nulla értéket. Emiatt az $[a, b]$ intervallumban $g'(x)$ -nek legalább $n + 1$ zérushelye van. A gondolatmenetet megismételve az adódik, hogy akkor $g''(x)$ -nek $[a, b]$ -ben legalább n zérushelye van stb. Így belátható, hogy van olyan $\xi \in [a, b]$, hogy

$$g^{(n+1)}(\xi) = f^{(n+1)}(\xi) - \frac{(n+1)!}{\omega_n(\hat{x})}(f(\hat{x}) - p_n(\hat{x})) = 0,$$

és innen a képlethibára azt kapjuk, hogy

$$f(\hat{x}) - p_n(\hat{x}) = \frac{\omega_n(\hat{x})}{(n+1)!} f^{(n+1)}(\xi).$$